

2 Random Numbers Generator

Full Rebuild Prompt — Exact Specification

React · TypeScript · Vite · Inline Styles · Inter Font

1. WHAT TO BUILD

A single-page React + TypeScript + Vite web app called "2 Random Numbers Generator". No backend, no Tailwind utility classes in the component — all styling is done with plain React inline styles. The app shows two dice-like number boxes side by side. Clicking the button generates two random integers from 1–6, animates them shaking for 600 ms while rapidly cycling through random values, then settles on the final result and shows a total sum badge.

Tech stack

- React 18 + TypeScript
- Vite (development server)
- + build
- All components styled: plain

React inline
style objects

—
no Tailwind
classes
session
App.tsx

•
Font: Inter,
loaded from
the system
/ browser

default stack (font Family: "Inter", sans-serif) • One CSS keyframe animation injected via a <style> tag in si

d
e
th
e
J
S
X

2. VISUAL DESIGN & COLOUR PALETTE

The app is light-themed. All colours are literal hex / rgba values applied as inline styles. There are no CSS variables, no dark mode, no theme tokens.

Colour reference

#f3eeff !' #e8d9fa Page background gradient (135deg, light lavender to soft purple)
#ffffff Card / panel background; also die face background
#3b1f6e Heading text ("2 Random Numbers Generator")
#9b7ec8 Subtitle text ("Click the button to generate"); also light purple accents
#6B3FA0 Number inside each die face (bold purple)
#e2d9f3 Die border (solid 2 px)
#f5f0fc Empty die placeholder background (before first roll)
#d4b8f7 Empty die placeholder border (dashed 2 px)
#c4a8f0 Light purple — "+" separator between dice; button bg when disabled/rolling
#7c3aed !' #a855f7 Total badge background gradient (135deg); also Generate button gradient
rgba(255,255,255,0.75) Total badge "TOTAL" label text
rgba(255,255,255,0.65) Total badge breakdown text (e.g. "3 + 4")
#ffffff Total badge number + button label text

Shadows

Page card 0 16px 60px
rgba(99,60,180,0.13), 0 2px 8px
rgba(0,0,0,0.07)
Die face 0 6px 24px
rgba(99,60,180,0.18), 0 2px 6px
rgba(0,0,0,0.10)
Total badge 0 4px 16px
rgba(124,58,237,0.25)
Button 0 4px 16px
rgba(124,58,237,0.30) (removed when rolling/disabled)

```
alignItems: "center",
gap: 36,
justifyContent: "center",
width: 300,
fontFamily: "'Inter', sans-serif",
}
```

3. PAGE LAYOUT

Inside the card, from top to bottom with gap: 36 between each section:

The outermost wrapper fills the full viewport and centres one child both horizontally and vertically:

H
e
a
d
e
r
b
l
o
c
k
(t
itl
e
+
s
u
b
t
itl
e)
•
Di
c
e
r
o
w
(t
w
o
di
e
b
o
x
e
s
wi
th

a
+
b
et
w
e
e
n,

O
R
tw
o
pl
a
c
e
h
ol
d
er
b
o
x
e

s
b
ef
or
e
fir
st
r
oll
)
•
R
e
s
ul
t
bl
o
ck
(t
ot
al
b
a
d
g
e
O
R
"
G
e
n
er
at
in
g
...
"
b
o
x
—
o
nl
y
o
n
e
s
h
o
w
n
at

a
ti
m
e)
•
G
e
n

```

er
at
e
/
G
e
n
er
function Die({ value, rolling }: { value: number; rolling: boolean }) {
  return (
    <div
      className={`die ${rolling ? "rolling" : ""}`}
      style={{
        width: 110,
        height: 110,
        borderRadius: 18,
        background: "white",
        boxShadow: "0 6px 24px rgba(99,60,180,0.18), 0 2px 6px rgba(0,0,0,0.10)",
        display: "inline-flex",

```

4. HEADER BLOCK

```

<h1> text "2 Random Numbers"
Generator
margin 0
fontSize 26
fontWeight 700
color #3b1f6e
letterSpacing 0.5px
<p> text "Click the button to generate"
margin "6px 0 0"
fontSize 14
color #3b1f6e

```

5. DIE COMPONENT <Die value rolling />

```

Box size 110 x 110 px
Border radius 18 px
Border 2 px solid #e2d9f3
Background white
Number size fontSize 52,
fontWeight 800
Number colour #6B3FA0
Number leading 1
<div style={{ display: "flex", gap: 28, alignItems: "center", minHeight: 110 }}>

```

The dice area always takes up the same vertical space (minHeight: 110) so the card does not jump when switching between pre-roll and post-roll states.

Before first roll — placeholder boxes

Two empty placeholder boxes, styled as dashed outlines:
No number, no shadow — just the pale lavender fill with a dashed purple border. Rendered in a flex row with gap: 28 (same as the real dice row) but WITHOUT the + separator.

After roll — two Die components + separator

When dice state is set: <Die value={dice[0]} rolling={rolling} /> then a + span then <Die value={dice[1]} rolling={rolling} />.

+ separator style fontSize: 28, color: "#c4a8f0", fontWeight: 300


```
{setInterval(() => null, 0)} // final answer, the final answer is rolled fresh inside the setTimeout callback.
// no + sign
// two placeholder dashed boxes - no + sign
// The interval this is processing is not in a useRef. This is safe because the setTimeout that
// least interval times should be used. It is a useRef. No else style in useEffect is
// needed because rolling guards re-entry.
// Generating... muted box
// Dice row wrapper (verbatim)
// Dice area
// The dice row wrapper has minHeight: 110. This prevents the card from shrinking when switching from the
// placeholder boxes (no + sign) to the real dice row. Without it the card would jump in height on the first roll.
```